

Deadlock Analysis and Prevention

Name: _____

ID: _____

Course: Operating Systems

Problem Statement

A multinational software company operates a centralized computing server where multiple applications execute simultaneously. The server has the following limited resources:

- 2 database connections: DB1 and DB2
- 2 network printers: PR1 and PR2
- 1 backup storage device: BS1

During peak business hours, four applications execute concurrently:

- Process P1 requires one database connection and one printer.
- Process P2 requires one printer and the backup storage device.
- Process P3 requires one database connection and the backup storage device.
- Process P4 requires one printer and one database connection.

The observed allocation is:

Process	Resources Currently Allocated	Resources Waiting For
P1	DB1	PR1
P2	PR1	BS1
P3	BS1	DB2
P4	DB2	DB1

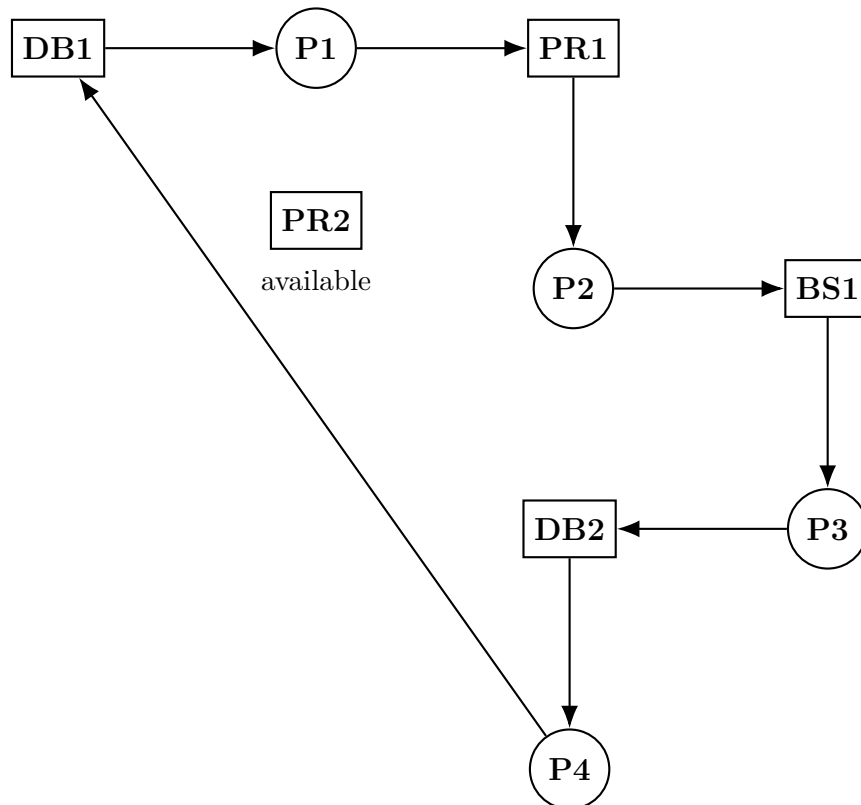
No process releases its allocated resource while waiting for another resource.

Assumptions

The table identifies exact resource instances. Therefore, if a process is waiting for PR1, it is waiting specifically for PR1 rather than any printer. PR2 is shown as an available printer instance, but it cannot satisfy P1's current request unless the system treats PR1 and PR2 as interchangeable. The main analysis follows the given table strictly and treats each listed instance as a distinct resource.

1. Resource Allocation Graph

In a resource allocation graph, process nodes are circles and resource-instance nodes are rectangles. A resource-to-process arrow represents an allocated resource. A process-to-resource arrow represents a pending request.



The graph contains the cycle:

$$\mathbf{P1 \rightarrow PR1 \rightarrow P2 \rightarrow BS1 \rightarrow P3 \rightarrow DB2 \rightarrow P4 \rightarrow DB1 \rightarrow P1}$$

Because every resource in this cycle is a single allocated instance, the cycle is sufficient evidence of deadlock for the exact resource-instance interpretation.

2. Allocation and Request Matrix

The same state can be represented in matrix form. A value of 1 means that the process either holds or requests that resource instance.

Allocation	DB1	DB2	PR1	PR2	BS1
P1	1	0	0	0	0
P2	0	0	1	0	0
P3	0	0	0	0	1
P4	0	1	0	0	0

Request	DB1	DB2	PR1	PR2	BS1
P1	0	0	1	0	0
P2	0	0	0	0	1
P3	0	1	0	0	0
P4	1	0	0	0	0

The available vector is:

DB1	DB2	PR1	PR2	BS1
0	0	0	1	0

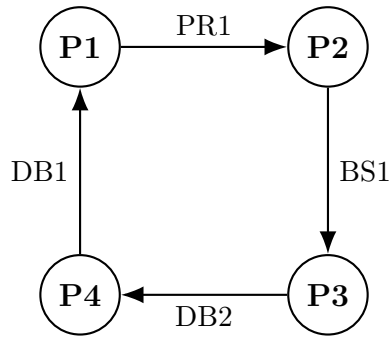
Only PR2 is available. However, no listed process is waiting for PR2. Therefore, no process can complete from the current state under the exact-instance request model.

3. Deadlock Analysis

A deadlock exists when a set of processes is permanently blocked because every process in the set is waiting for an event that only another process in the same set can cause. Here, the waiting chain is:

- P1 holds DB1 and waits for PR1, which is held by P2.
- P2 holds PR1 and waits for BS1, which is held by P3.
- P3 holds BS1 and waits for DB2, which is held by P4.
- P4 holds DB2 and waits for DB1, which is held by P1.

This can be simplified into a wait-for graph:



The wait-for graph contains the cycle $P1 \rightarrow P2 \rightarrow P3 \rightarrow P4 \rightarrow P1$. Since none of these processes can release its current resource while waiting, the system cannot move to a state where any process in the cycle finishes. Therefore, the observed state is a deadlock.

Safe Sequence Check

A safe sequence exists only if at least one process can obtain its remaining requested resource, finish, and release its allocated resource. In this state:

- P1 cannot finish because PR1 is unavailable.
- P2 cannot finish because BS1 is unavailable.
- P3 cannot finish because DB2 is unavailable.
- P4 cannot finish because DB1 is unavailable.

Thus, there is no process that can be selected as the first process in a safe sequence. The state is unsafe and, under the stated no-release condition, deadlocked.

4. Four Necessary Conditions for Deadlock

Mutual Exclusion

At least one resource must be non-shareable. In this case, each database connection, printer instance, and backup storage device can be assigned to only one process at a time. The backup storage device BS1 is especially restrictive because only one instance exists. This condition is satisfied.

Hold and Wait

Each process holds one resource while requesting another. P1 holds DB1 while waiting for PR1, P2 holds PR1 while waiting for BS1, P3 holds BS1 while waiting for DB2, and P4 holds DB2 while waiting for DB1. This condition is satisfied.

No Preemption

The system does not forcibly remove resources from waiting processes. The problem explicitly says that no process releases its allocated resource while waiting. This means the operating system cannot break the cycle by taking DB1, PR1, BS1, or DB2 away from the current holders. This condition is satisfied.

Circular Wait

The processes form a closed waiting chain: P1 waits for P2, P2 waits for P3, P3 waits for P4, and P4 waits for P1. This condition is satisfied.

Since all four necessary conditions are present simultaneously, the deadlock is confirmed.

5. Prevention and Avoidance Techniques

Deadlock Prevention

Prevention designs the system so at least one necessary condition can never occur.

- **Eliminate mutual exclusion:** This is usually not practical for physical or limited resources. Printer requests may be handled through spooling, but database connections and backup storage still require exclusive control.
- **Eliminate hold and wait:** Each process must request all required resources before it starts. For example, P2 would request PR1 and BS1 together. This prevents deadlock, but it wastes resources because a process may hold a resource long before it actually needs it.
- **Allow preemption:** If a process cannot obtain its next resource, the system can roll back the process and reclaim its resources. This works better for CPU and memory than for printers, storage operations, or database transactions, because those may require careful rollback.
- **Break circular wait:** Define a fixed resource order and require every process to request resources in that order. For example, all applications could request database connections first, printers second, and backup storage last. If all processes follow this order, a circular wait cannot form.

Deadlock Avoidance

Avoidance makes a decision at request time. The system grants a request only if the resulting state remains safe.

- **Banker's algorithm:** The operating system compares available resources with each process's maximum possible need. A request is granted only if a safe completion sequence still exists. This method is strong, but it requires accurate maximum demand information.
- **Resource-allocation graph with claim edges:** The system records possible future requests and avoids granting a request that could produce an unsafe cycle. This is useful for single-instance resources.
- **Admission control:** The server can limit how many applications enter the critical resource-acquisition phase at once. This reduces the chance of all limited resources being held by incomplete processes.
- **Timeout with rollback:** A process that waits too long can be aborted, rolled back, and restarted later. This is a recovery-oriented technique and is useful in real servers, but by itself it does not prove that deadlock can never occur.

6. Comparative Evaluation

Technique	Strength	Weakness	Suitability
Request all resources at once	Simple and guarantees no hold-and-wait deadlock	Low utilization and possible starvation	Medium
Resource ordering	Low overhead and directly prevents circular wait	Requires consistent design across all applications	High
Preemption and rollback	Can recover when a process blocks too long	Hard for database, printing, and backup operations	Medium
Banker's algorithm	Provides a formal safe-state check	Requires known maximum demands and runtime checks	Medium
Admission control	Reduces contention during peak hours	May reduce throughput when demand is high	Medium
Timeout and retry	Practical and easy to monitor	Recovery method, not complete prevention	High as support

7. Recommendation

The best solution for this server is a combination of **global resource ordering**, **request validation**, and **timeout-based rollback**.

The primary prevention rule should be:

Database connections → Printers → Backup storage

Every process must request resources only in this order. For example, a process that needs both a printer and the backup device must acquire the printer before BS1. A process that needs both a database connection and a printer must acquire the database connection before the printer. This breaks circular wait, which is the most practical necessary condition to eliminate in this scenario.

The server should also maintain a resource manager that checks each request before granting it. If the request violates the global ordering rule, it should be denied or delayed. If a process waits beyond a fixed timeout, the system should roll back the process, release its held resources, log the event, and restart the process later. This makes the design both theoretically correct and operationally practical.

Conclusion

The given allocation produces a circular wait among P1, P2, P3, and P4. The allocation matrix, request matrix, resource allocation graph, and wait-for graph all show that no process can proceed first. Because mutual exclusion, hold and wait, no preemption, and circular wait are all present, the system is deadlocked. The strongest practical solution is to enforce a global resource-ordering policy, supported by request validation and timeout-based rollback.